

Instituto Tecnológico de Costa Rica
Escuela Ingeniería en Computadores
Curso: CE1104 Fundamentos de sistemas computacionales

Documentación técnica

StrangerTEC

Estudiantes:

Ignacio José Apuy Anchía
Jordanny Steven Hernández Mora

Profesor:

Leonardo Araya Martínez

Semestre I 2026

1. Introducción	3
1.1. Propósito del documento	3
1.2. Descripción del proyecto	3
1.3. Objetivos y Alcance	4
1.4. Definiciones, Acrónimos y Abreviaturas	4
2. Descripción General del Sistema	5
2.1. Perspectiva del Producto	5
2.2. Funcionalidades principales	5
2.3. Restricciones del sistema, Supuestos y Dependencias	5
3. Requisitos del Sistema	6
3.1. Requisitos Funcionales	6
3.2. Requisitos No Funcionales	6
4. Diseño de la Solución	7
4.1. Visión General de la Arquitectura	7
4.2. Descripción de Componentes	8
4.3. Stack Tecnológico	9
4.4. Diagrama de Clases y Modelo de Datos	9
4.5. Diagrama de Flujo y Secuencia	11
4.6. Decisiones de Diseño y Justificación	13
5. Implementación y Validación de Calidad	14
5.1. Estructura del repositorio	14
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución	14
5.3. Estrategias y Casos de Prueba	15
5.4. Resultados de pruebas y problemas encontrados	16
6. Declaración de uso de Inteligencia Artificial y herramientas	23
6.1. Herramientas de IA utilizadas	23
6.2. Descripción del Uso de Inteligencia Artificial	23
6.3. Declaración de Responsabilidad y Autoría	24
7. Conclusión	25
8. Bibliografía	26

1. Introducción

StrangerTEC es un proyecto de propósito investigativo en el que se plantea aplicar técnicas y métodos para elaborar un prototipo basado en un aspecto de ficción proveniente de la serie Stranger Things.

El aspecto como tal consiste en un mural de letras y luces, donde estas últimas se encienden en patrones específicos con la intención de transmitir un mensaje de forma visual desde otro mundo, mientras los personajes responden y hacen preguntas de forma verbal hacia el otro lado.

Para el desarrollo del proyecto se propuso recrear este mismo comportamiento mediante conceptos, sistemas eléctricos y programación de microcontroladores, quienes en conjunto generan una idea de cómo podría funcionar esta característica presentada en la serie, aplicando conocimientos en electrónica, programación y confección, efectivamente replicando esta mecánica con similaridad dentro de lo que es posible.

1.1. Propósito del documento

El presente documento contiene información esencial del desarrollo, medidas y decisiones tomadas para el cumplimiento de los objetivos e idea general propuesta. El documento está adaptado a investigadores y evaluadores para que accedan y entiendan con facilidad las ideas planteadas y descritas durante el proceso.

1.2. Descripción del proyecto

Este proyecto describe un sistema el cual se centra en la utilización de microcontroladores programables y su interacción con diferentes componentes electrónicos. En este caso el microcontrolador empleado forma parte de la serie de Arduino, donde es más específicamente un Arduino Mega 2560. Este dispositivo cuenta con la capacidad de ser programado con diferentes instrucciones para enviar y recibir datos por medio de pulsos y/o señales eléctricas a los diferentes componentes que se le añadan, sean estos luces LED, botones, y demás. El prototipo tiene como función adicional, brindar una idea de un juego donde se interactúa con la maqueta y otros usuarios.

Se destina el proyecto a aquellos quienes posean interés en obtener conocimientos sobre la ingeniería y metodologías utilizadas, el concepto detrás de la idea propuesta o simplemente requieran analizar perspectivas e interpretaciones distintas de una misma idea.

1.3 Objetivos y Alcance

- Objetivo general: Desarrollar un prototipo utilizando microcontroladores y componentes electrónicos, así como modos de juego interactivos y otros conceptos, emulando un comportamiento similar al del muro de luces presentado en la serie Stranger Things.
- Objetivos específicos:
 - Crear un prototipo funcional y autónomo en base al concepto propuesto del muro de luces.
 - Diseñar diferentes modos de juego donde se evidencie y se demuestre la idea principal de forma dinámica e interactiva.
 - Diseñar un sistema de rondas y puntajes donde se mide la destreza de el o los usuarios.
- Alcance: Se limita el funcionamiento del prototipo a dos modos de juego simples, partidas cortas, repertorio de frases limitado y ciclos de juego cerrados y comunicación mediante monitor serial a la computadora.

1.4 Definiciones, Acrónimos y Abreviaturas

- Componentes eléctricos: Piezas creadas con un funcionamiento específico en base a señales eléctricas que los encienden, apagan, generan resultados, y demás, donde existen para un gran número de aplicaciones para la vida cotidiana.
- LED: Tecnología de luces blancas o de colores que se encienden o apagan mediante pulsos eléctricos.
- Buzzer: Componente que genera ruido en diferentes frecuencias.
- Circuito integrado (IC): Es un componente eléctrico designado para una tarea específica como redirigir pulsos a diferentes salidas posibles.
- Microcontrolador: Dispositivo designado al control e interfaz con diferentes componentes compatibles con el mismo. Es programable y modular, o sea que puede gestionar tareas, acciones y respuestas con otros componentes a los que esté conectado.
- Arduino: Marca registrada y reconocida que elabora microcontroladores potentes y capaces de ser adaptados a gran cantidad de tipos y calibres de proyecto.
- C++: Lenguaje de programación de bajo nivel empleado y adaptado en el ecosistema Arduino para programar sus funciones y tareas específicas a realizar.
- Código Morse: Simbología utilizada para simplificar el uso de letras y reducirlo a puntos o rayas, donde ambos representan pulsos de diferente duración que pueden ser interpretados para recuperar el mensaje original.

2. Descripción General del Sistema

En esta sección se brindan detalles esenciales del funcionamiento interno y las metodologías utilizadas por y para el producto como resultado de este proyecto.

2.1. Perspectiva del Producto

El sistema se crea a partir del microcontrolador Arduino, el cual es programado con un código fuente en C++ para las tareas específicas y control entre los componentes, para satisfacer la idea principal que se desea aplicar.

2.2. Funcionalidades principales

- Interacción y comunicación: Indicaciones y guías efectivas para que el usuario entienda cómo utilizar adecuadamente el producto.
- Manejo de partidas: Opciones de modos de juego disponibles, manejo de jugadores y estadísticas, manejo de errores.
- Control de componentes: Coordinación con y entre los componentes externos.

2.3. Restricciones del sistema, Supuestos y Dependencias

- Restricciones:
 - El sistema debe utilizar un voltaje de entre 5 y 12 voltios para su correcto funcionamiento.
 - El sistema no puede ejecutarse sin un monitor serial en una computadora.
- Supuestos:
 - El usuario tiene conocimiento sobre Arduino.
 - Se tiene instalado Arduino IDE 2.x.
 - Los componentes y sus conexiones están en buen estado.
 - El prototipo está conectado mediante USB a la computadora con el monitor serial abierto.
 - El usuario tiene conocimiento básico sobre código Morse.
- Dependencias:
 - Librería wdt.h para el manejo de reinicios.
 - Cable USB tipo A-B para conectar a la computadora.
 - Fuente de alimentación (Puede utilizarse del mismo cable).

3. Requisitos del Sistema

En esta sección se presentan diferentes requisitos funcionales y no funcionales del sistema. Ambos son necesarios para definir las capacidades del programa y su comportamiento deseado.

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	Comunicación maqueta-computadora	Alta	Están adecuadamente conectadas la computadora y la maqueta, y el monitor serial está recibiendo datos del Arduino.
RF-002	Detección de pulsos	Alta	El Arduino detecta adecuadamente los pulsos enviados desde el botón.
RF-003	Encendido de LEDs	Media	Las LEDs se encienden correctamente y en el orden adecuado.

3.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Integridad	La maqueta se encuentra en buen estado, donde todos sus componentes y el cableado se encuentran bien conectados y en orden.	Funcionamiento total del prototipo.
RNF-002	Comunicación	El arduino es capaz de conectarse, enviar y recibir mensajes de la computadora.	Correcto traslado de información entre la computadora y el prototipo.
RNF-003	Accesibilidad	Las instrucciones son claras y orientan al usuario en el adecuado uso del prototipo.	El usuario es capaz de entender cómo funciona y cómo se debe utilizar el prototipo.

4. Diseño de la Solución

En este capítulo se destacan las decisiones, entornos y modelos de desarrollo utilizados para la ejecución del proyecto.

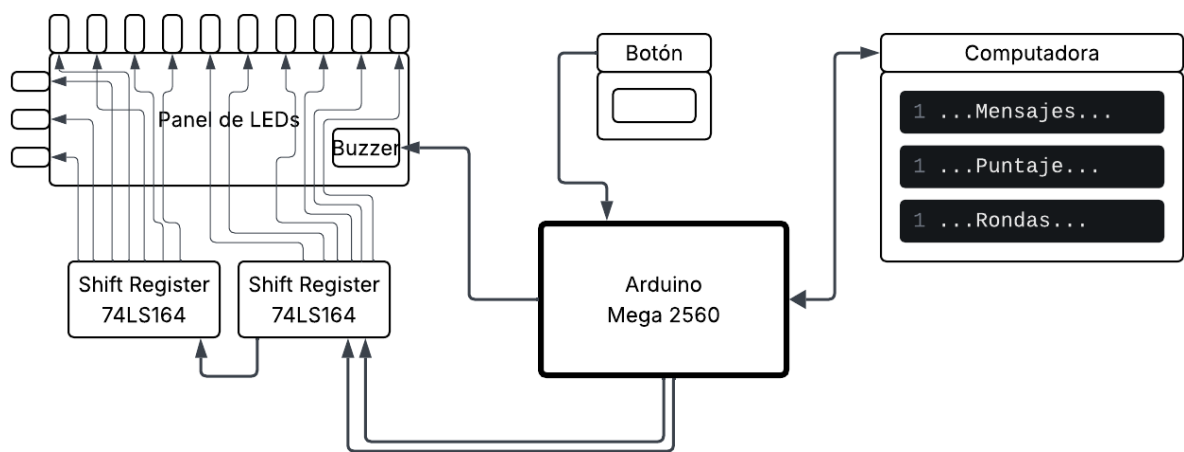
4.1. Visión General de la Arquitectura

El sistema de este proyecto utiliza una arquitectura monolítica. En este tipo de arquitectura se utiliza un modelo centralizado y encargado de administrar toda una única ejecución. El Arduino posee un único código fuente y a su vez se encarga de comunicarse con los componentes y la computadora, mas si llega a suceder que el Arduino se detiene, toda la ejecución lo hace también.

Se tomó este diseño debido a que el Arduino implica utilizarlo dado que está hecho ya de esta forma, además de que el propósito general del prototipo es uno único y específico, por lo que es posible prescindir de otros módulos.

Seguido de esto, se presenta un diagrama sobre la arquitectura utilizada y su estado dentro del proyecto. Los elementos son los siguientes:

- **Arduino Mega 2560:** El procesador principal de todo el prototipo. Se encarga de procesar datos provenientes del botón, esperar acciones en la computadora y encender LEDs en el panel cuando sea necesario, enviando cadenas a los shift register.
- **Botón:** Deja pasar una corriente asignada cuando está pulsado, y la interrumpe cuando no lo está.
- **Shift Register 74LS164:** Son circuitos integrados (IC) que se encargan de convertir cadenas binarias (Ej: 0101) en salidas de corriente para cada conexión. Se conectan en cadena para administrar cadenas más largas y conmutar LEDs de forma paralela.
- **Panel de LEDs:** Montaje de diferentes LEDs en forma de tabla, sobre una placa de pruebas (Protoboard), donde cada LED se enciende según su cable asignado. La forma de tabla se utiliza para mostrar una letra en una celda específica utilizando dos LED.
- **Buzzer:** Conocido como zumbador, produce un sonido. Este sonido se reproduce cada vez que se pulsa el botón o se muestra una letra en el panel a modo de respuesta sonora.
- **Computadora (Terminal Serial):** Se utiliza el monitor serial desde la computadora para recibir mensajes desde el Arduino e ingresar datos para continuar.



4.2. Descripción de Componentes

Componente	Responsabilidad	Tecnología	Dependencias
Microcontrolador	Se encarga de procesar toda la información entre los componentes y la computadora y ejecutar tareas.	Arduino Mega 2560	Fuente de energía. Conexión USB para enviar instrucciones.
Computadora	Muestra instrucciones, puntajes, estados y procesos del juego.	Windows 11	Arduino IDE 2.x
Shift Registers	Reciben cadenas binarias y dejan pasar corriente según lo que hayan recibido.	74LS164	Fuente de energía y entrada de datos.
Panel de luces y buzzer.	Retroalimentación visual y sonora. Entrada para las interacciones.	LED, buzzer, botón, etc.	Fuente de energía, cableado hacia el Arduino.

4.3. Stack Tecnológico

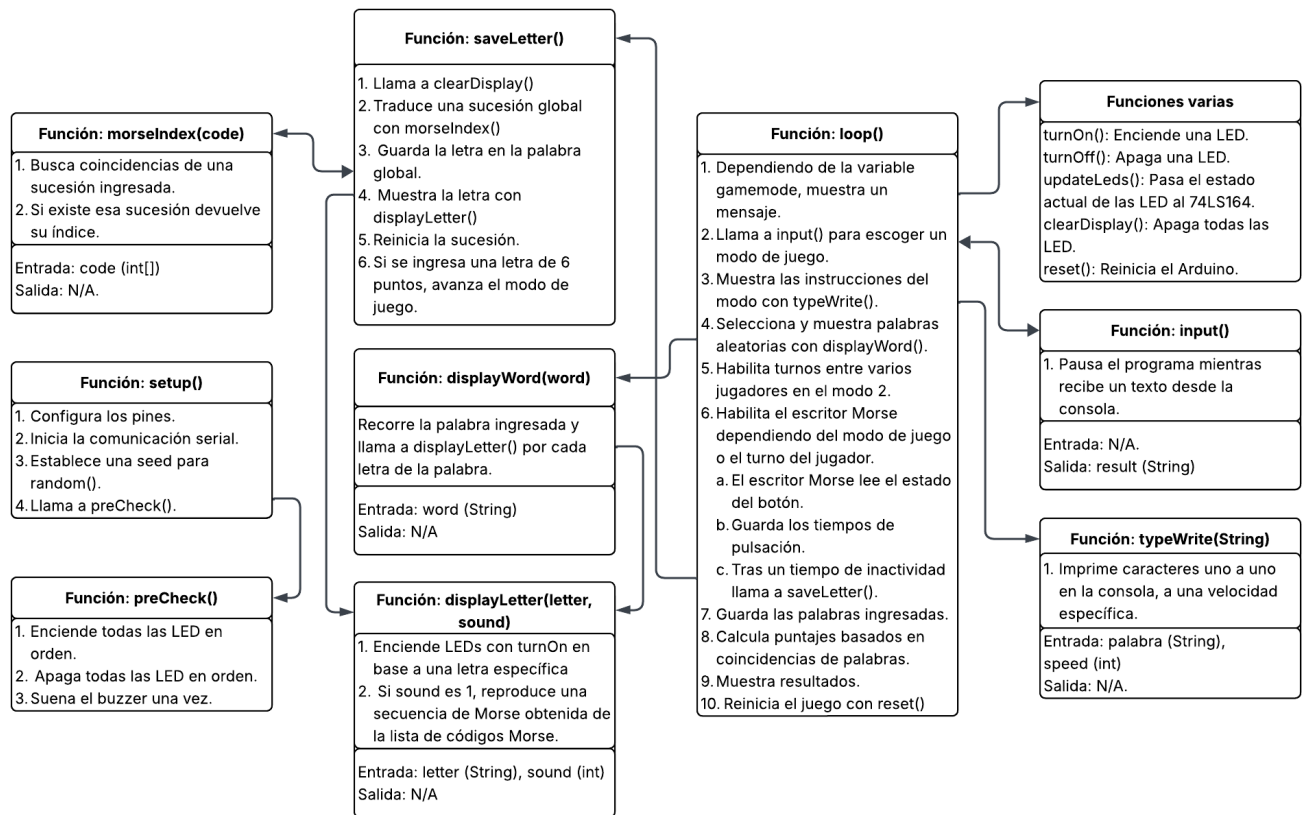
Categoría	Tecnología	Versión	Propósito
Microcontrolador	Arduino Mega 2560	R3	Procesamiento central y control de entradas y salidas.
Editor de código (IDE)	Arduino IDE	2.3.8	Programación, compilación, depuración y comunicación.
Lenguaje de programación	C++	Estándar	Lógica y sintaxis para las instrucciones del prototipo.
Librerías	avr/wdt.h	Estándar	Manejo de reinicios de hardware.

4.4. Diagrama de Clases y Modelo de Datos

El siguiente es un diagrama de funciones que detalla cada parte del código fuente del prototipo y cómo se relacionan los datos y las secuencias dentro del mismo. Cada función tiene propósitos específicos como

- `turnOn()`, `turnOff`, `updateLeds()`, `clearDisplay()`: Se encargan de modificar la cadena de bits, ya sea apagando o encendiendo campos antes de pasarla al circuito integrado 74LS164 que encenderá las LED en base a la hilera recibida.
- `reset()`: Reinicia el Arduino utilizando `wdt.h`.
- `preCheck()`: Enciende y apaga los componentes al iniciar para revisar que todo funcione correctamente.
- `typeWrite()`: Escribe letra a letra a modo de animación al mostrar diálogos.
- `input()`: Detiene el programa temporalmente hasta recibir una entrada desde el monitor serial en la computadora.
- `displayLetter()`: Busca en un diccionario una letra ingresada y enciende LEDs específicas para indicar cuál letra es.
- `displayWord()`: Recibe una palabra y por cada letra, las muestra con `displayLetter`.
- `saveLetter()`: Recibe la cadena actual de pulsos y la traduce a una letra utilizando `morseIndex()`, para luego mostrar esa letra y crear una palabra global con cada letra resultante de esta función. Además si se ingresa una letra con 6 puntos, se avanza la fase del modo de juego actual.
- `morseIndex()`: Traduce cadenas de pulsos a letras mediante índices y comparaciones.
- `setup()`: Se ejecuta al iniciar el Arduino. Configura pines y variables, inicia la comunicación serial y llama a `preCheck()`.

- `loop()`: Se ejecuta en bucle después de `setup()`. Este se encarga de mostrar diálogos con `typeWrite()`, pedir entradas al usuario con `input()`, avanzar fases de juego, mostrar turnos de jugadores en el monitor serial, guardar palabras, calcular puntajes, habilitar el escritor de Morse donde se reciben pulsos y se llama a `saveLetter()`, mostrar resultados en el monitor serial y finalmente reiniciar el juego con `reset()`.



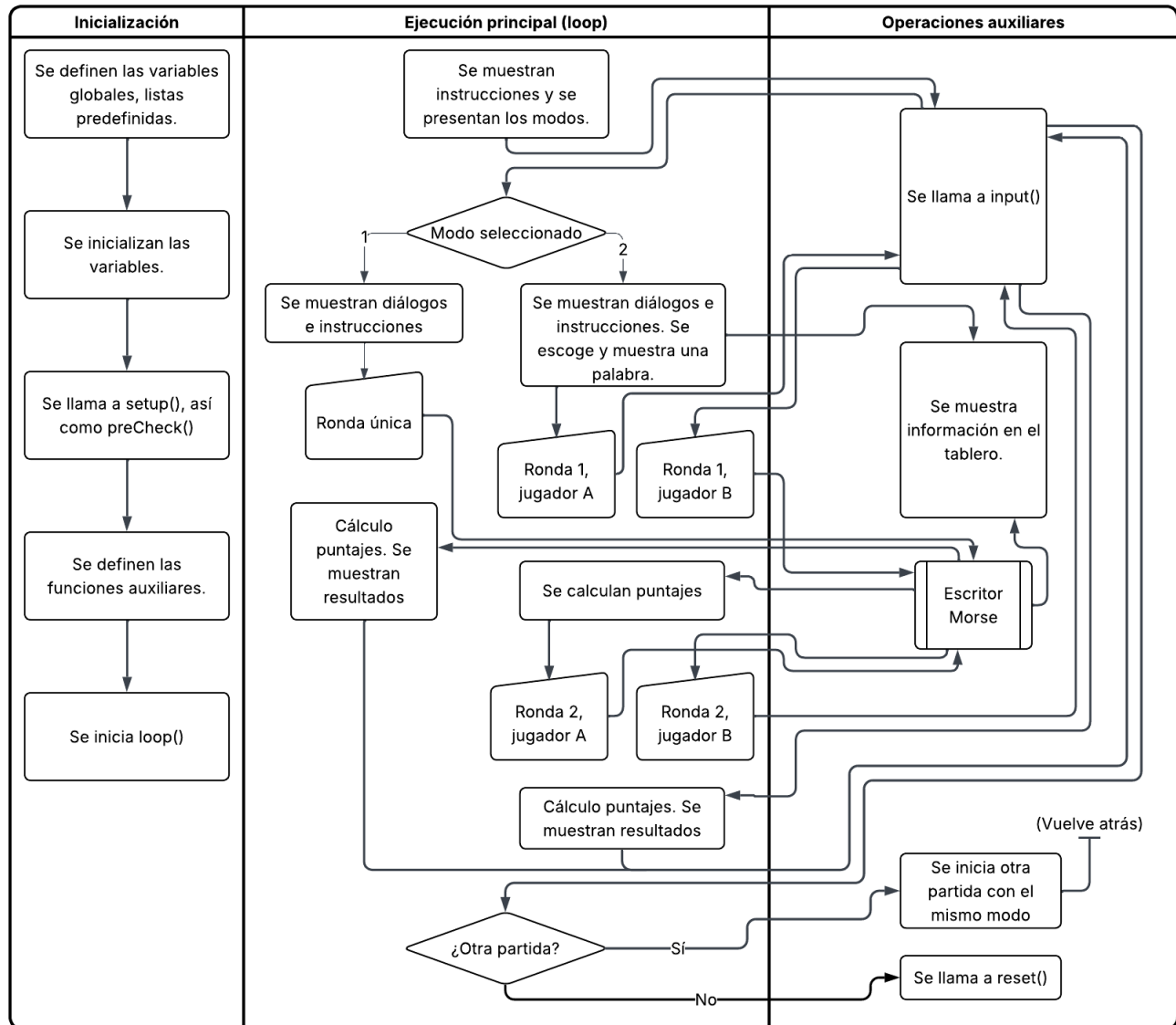
4.5. Diagrama de Flujo y Secuencia

Incluye diagramas que representan los flujos de proceso más importantes (UML: diagramas de secuencia, actividades o casos de uso). Documenta al menos los flujos críticos del negocio.

Se debe describir brevemente cada diagrama: qué flujo representa y cuáles son los actores o sistemas involucrados.

El siguiente es un diagrama de flujo del funcionamiento del sistema durante una ejecución total, así como los diferentes actores y sus efectos sobre las secuencias. Algunos de los procesos descritos son:

1. Se inicia desde la columna izquierda donde se comienza a ejecutar el código, definiendo e inicializando variables globales y listas que se utilizarán luego.
2. Se llama `setup()`, el cual pines y el monitor serial.
3. Se definen funciones y finalmente se inicia el loop principal.
4. En el loop, se comienza dando la bienvenida con diferentes diálogos que utilizan `typeWrite()`, y luego se le pide al usuario seleccionar un modo entre 1 y 2.
5. En el modo 1:
 - a. Se muestran las instrucciones y se inicia el escritor Morse para que el usuario ingrese una palabra.
 - b. Una vez finalizado, se devuelve a la ejecución donde se compara la palabra con otras similares y se da un puntaje según los aciertos.
 - c. Se muestran los resultados y se le pide al usuario si desea iniciar otra partida o reiniciar.
 - d. Si se inicia otra partida simplemente el modo de juego vuelve atrás, después de seleccionar el modo.
 - e. Si se reinicia, vuelve a comenzar todo el programa desde cero.
6. En el modo 2:
 - a. Se muestran las instrucciones y se selecciona una palabra aleatoria, la cual se muestra en el tablero.
 - b. Se le pide al usuario A ingresar la palabra en la terminal.
 - c. Después de esto, se habilita el escritor Morse para el jugador B, el cual debe ingresar la palabra.
 - d. Se guardan ambas palabras y se calculan los puntajes respecto a aciertos por letra de la palabra original.
 - e. Se continua con otra ronda donde se habilita el escritor Morse para que el jugador A esta vez ingrese la palabra.
 - f. Seguidamente, se le pide al jugador B ingresar la palabra desde la consola.
 - g. Finalmente se calculan los puntajes con los nuevos intentos y se muestran los resultados y el ganador.
 - h. Se da la opción de iniciar otra partida en este modo o reiniciar todo el programa de 0.



4.6. Decisiones de Diseño y Justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Microcontrolador	Raspberry PI Arduino	Arduino	Es más que capaz de cumplir con los requisitos para hacer la idea principal, ya que provee las herramientas adecuadas.
Arquitectura	Monolítica Distribuida	Monolítica	Es suficiente para satisfacer el concepto debido a su gestión centralizada del ciclo de juego, además de que es la usual en proyectos de Arduino.
Lenguaje de programación	C++ Python	C++	Es el lenguaje de programación por defecto para Arduino. Es un lenguaje de alto rendimiento y su compilador es lo bastante potente como para evitar problemas graves.
Editor de código	Visual Studio Code Arduino IDE	Arduino IDE	Es el editor de código por defecto para Arduino, brindando un ambiente nativo del mismo, además de que permite comunicarse con él, compilar y enviar código fuente directamente.

5. Implementación y Validación de Calidad

En este capítulo se presenta la estructura del repositorio donde se encuentran los archivos de proyecto, así como los pasos para acceder y ejecutarlo. También se tiene el registro de las pruebas realizadas durante el proceso de desarrollo, y sus resultados.

5.1. Estructura del repositorio

Ruta	Archivo	Descripción
Proyecto_StrangerTEC/	ProyectoStrangerTEC.ino	Archivo compilable con la programación e instrucciones para el Arduino.
Proyecto_StrangerTEC/	Documentación Externa StrangerTEC.docx	Documentación externa del proyecto (Este archivo)
Proyecto_StrangerTEC/	Documentación de Atributos StrangerTEC.docx	Documentación de atributos y descripción del equipo de desarrollo.

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

- Entorno:
 - Sistema Operativo: Windows 11.
 - Microcontrolador: Arduino Mega/Uno.
 - Se requiere el editor de código Arduino IDE 2.x.
 - Se requiere Git para clonar el repositorio.
- Guía de instalación y ejecución:
 - 1. Instalación física (Hardware):
 - 1.1. Conectar los pines según el diagrama en la sección 4.1, donde los pines correspondientes se encuentran indicados en el código ProyectoStrangerTEC.ino.
 - 2. Instalación del código (Software):
 - 2.2. Abrir la terminal de Windows y clonar el repositorio con:
git clone https://github.com/NaxoW77/Proyecto_StrangerTEC.git
 - 2.2. Una vez clonado, abrir ProyectoStrangerTEC.ino utilizando Arduino IDE.
 - 2.3. Seleccionar el puerto donde está conectado el Arduino.
 - 2.4. Subir el código al microcontrolador Arduino.

5.3. Estrategias y Casos de Prueba

En esta sección se describen metodologías, pruebas y resultados para evaluar la calidad del prototipo como producto final. Se realizaron diferentes evaluaciones rutinarias y específicas para asegurarse del correcto funcionamiento y planteamiento adecuado del concepto de la idea principal.

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas Unitarias	Verificación de la funcionalidad de cada componente de hardware.	Manualmente	Funcionalidad correcta de los componentes de forma individual.
Pruebas de integración	Validar la funcionalidad de los componentes de hardware en conjunto con el código	Manualmente	Funcionalidad de los componentes como conjunto relativamente final.
Prueba de comunicación	Comprobar la comunicación entre el prototipo y una computadora	Arduino IDE Monitor Serial	Comunicación correcta entre prototipo y terminal
Prueba de aceptación (UAT)	Verificar el funcionamiento del prototipo final, en conjunto con la computadora	Arduino IDE Monitor Serial	Funcionalidad aceptable para usuario final. Modos de juego y ciclos de partida funcionales.

- Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-001	Inicio correcto del arduino	1. Conectar el prototipo a la computadora. 2. Utilizar el monitor serial	Se muestra información en el monitor y el arduino realiza el chequeo previo de funcionamiento de LEDs.	El prototipo ejecuta el chequeo y envía información al monitor serial.	Pasó
CP-002	Funcionamiento correcto de los LEDs	1. Encender el prototipo. 2. Verificar que las LEDs se enciendan durante la revisión.	Todos los LEDs se encienden al realizar el chequeo inicial.	Los LEDs se encienden en el orden programado en la revisión.	Pasó
CP-003	Reconocimiento de patrones del pulsador	1. Iniciar el minijuego. 2. Utilizar el pulsador para enviar señales.	Se recibe e interpreta el carácter correctamente según las pulsaciones ingresadas.	Algunas pulsaciones no se detectaron como se debía, provocando la salida de	No pasó

				caracteres no deseados.	
CP-00 4	Combinación correcta de LEDs en el abecedario del prototipo	1. Inicia el minijuego 2. Los LEDs se encienden y muestran letras.	La combinación de LEDs en fila y columna muestran el carácter correspondiente.	Los LEDs se encienden y señalan el carácter.	Pasó
CP-00 5	Funcionamiento correcto del prototipo dentro de la caja.	1. Introducir el arduino dentro de la caja que lo contiene. 2. Conectarlo a la computadora e iniciar su funcionamiento.	Correcto funcionamiento del módulo y posibilidad de lectura por el usuario.	El arduino funciona correctamente en la caja. Es posible leer la combinación de LEDs, utilizar el pulsador y conectarlo a la computadora.	Pasó
CP- 006	Resistencia al movimiento del prototipo.	1. Ensamblar el módulo en la caja 2. Comprobar resistencia al movimiento.	Todo el módulo debe mantenerse funcional ante movimientos leves.	El arduino se mantiene funcional a pesar de existir cierta movilidad.	Pasó

5.4. Resultados de pruebas y problemas encontrados

Resultados:

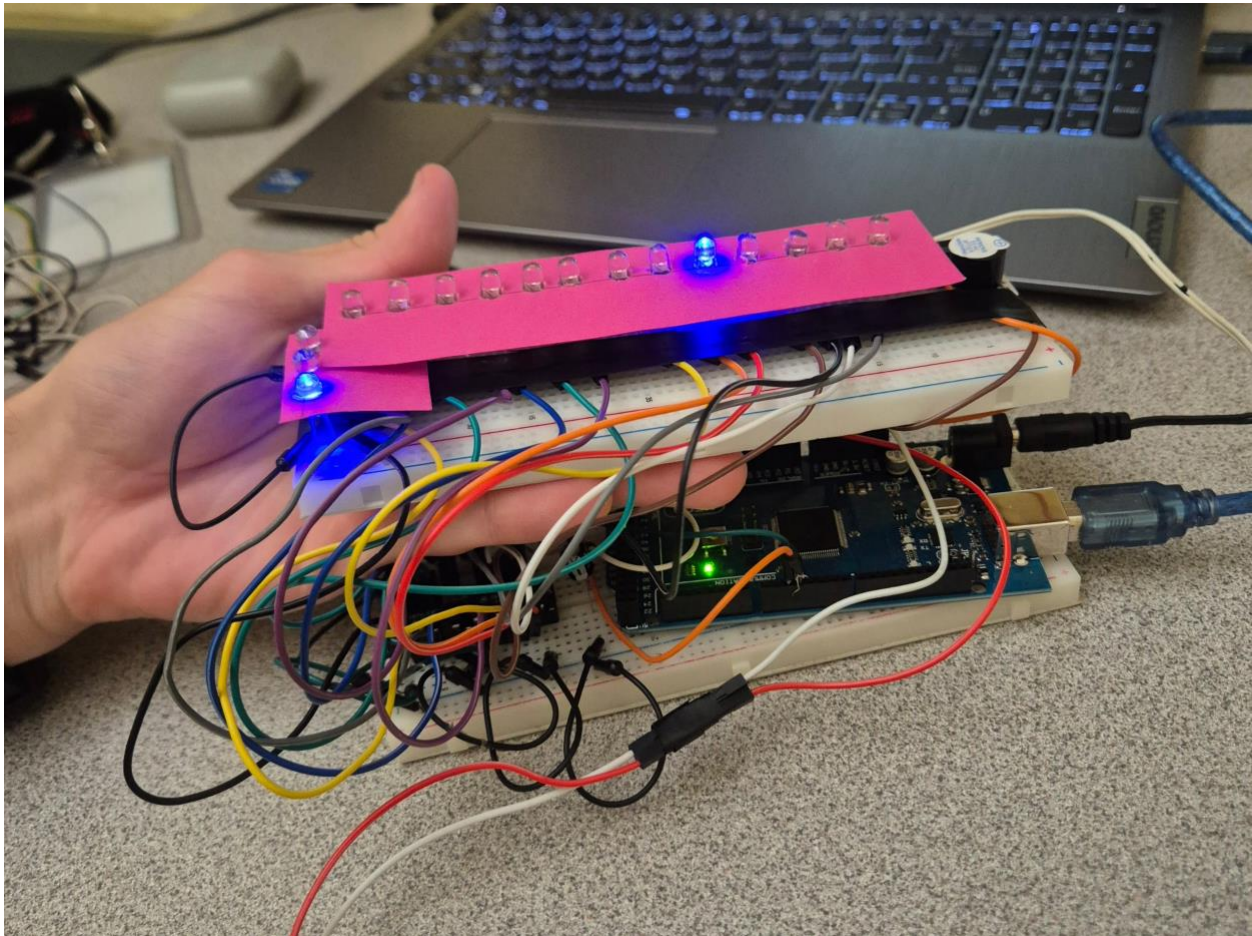
○ Casos de prueba exitosos:

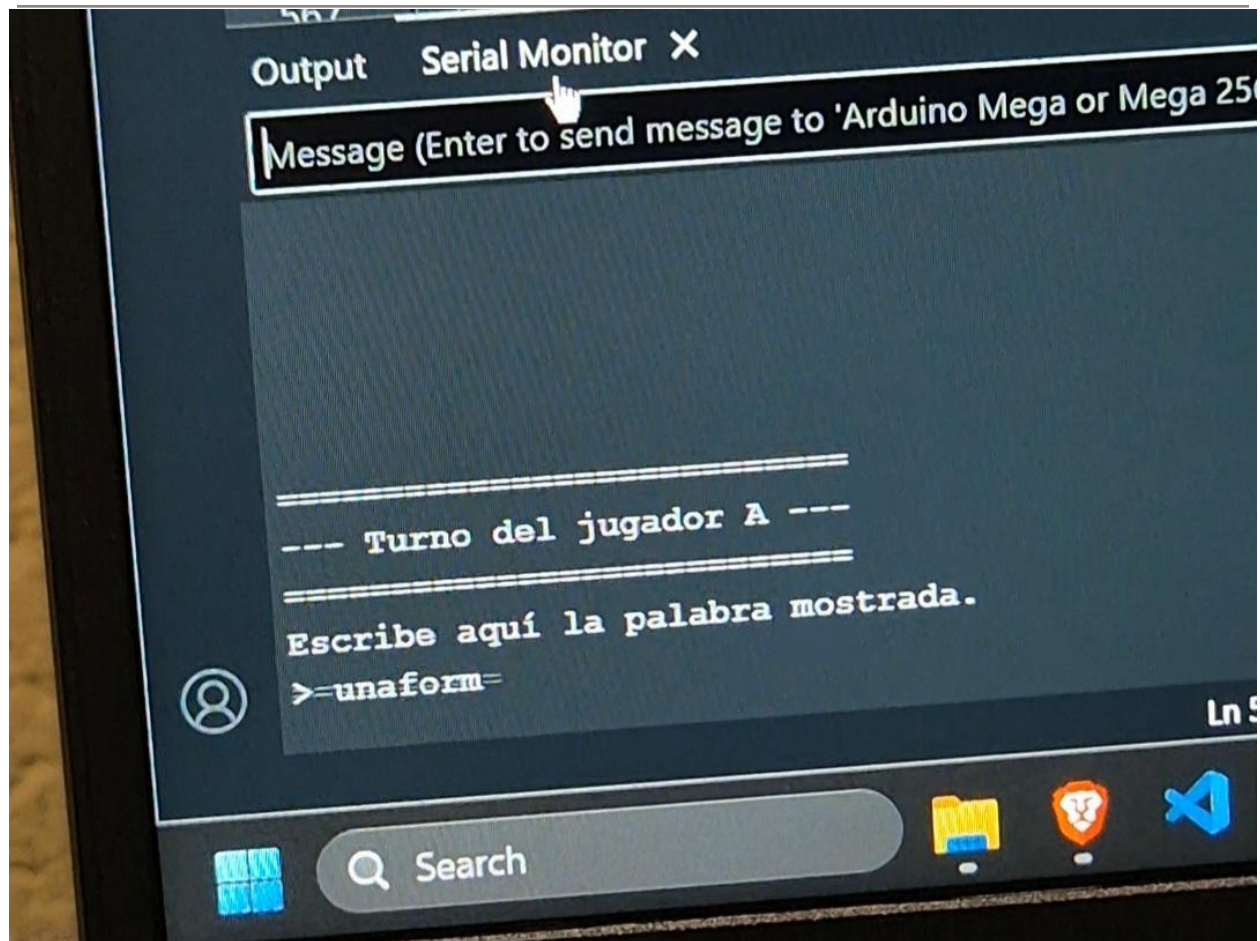
- **CP-001:** Se prueba que el arduino inicie correctamente al estar conectado a la computadora. Este mismo debe emitir un chequeo previo de los LEDs conectados para verificar su funcionamiento a su vez que envía información al monitor serial del IDE. Esta prueba concluye exitosamente, con el resultado previsto.
- **CP-002:** Se pretende verificar que los LEDs que utiliza el arduino funcionen correctamente cuando este inicia su programa. Este módulo utiliza un sistema de chequeo para verificar que todos los LEDs funcionan cada que inicia, en caso de que alguno no encienda es evidente que hay un problema. Finalmente todos los LEDs demostraron funcionar como es debido. Se concluye la prueba exitosamente.
- **CP-004:** Busca verificar que la combinación fila-columna de los LEDs correspondan con el carácter designado según las rúbricas y lo establecido en el código. A pesar de ser uno de los más grandes problemas con el prototipo, finalmente se resolvió. La prueba concluye exitosamente.
- **CP-005:** Pretende comprobar que todo el módulo de arduino funcione correctamente dentro de la caja que lo cubre y que siga cumpliendo su propósito principal para el usuario final. Para que se cumpla necesita que:
 - La combinación de LEDs en el alfabeto sea visible por el usuario.
 - El pulsador para código Morse sea accesible desde fuera
 - El cable que conecta el arduino a la computadora sea accesible.

- El sonido del buzzer puede ser escuchado.

Todo esto se cumple al finalizar la prueba, por lo que se concluye un caso de éxito en esta.

- **CP-006:** Pretende verificar que tan frágil puede ser la maqueta ante movimientos leves, y si esta se mantiene funcional a pesar de ello. Toda la maqueta mantiene su funcionamiento debido a sus componentes fijados. Es un caso de éxito para la prueba.
- Estas son algunas imágenes obtenidas durante las pruebas:






```
Arduino Mega or Mega 2560

ProyectoStrangerTEC.ino

224 // Se realiza la comprobación inicial de los componentes
225 preCheck();
226
227 }
228 // Función para mostrar una letra específica en el panel
229 void displayLetter(String letter, int sound = 0) {
230
231     // Se asegura que la letra esté en mayúsculas para evitar fa
232     letter.toUpperCase();
233
234     // Se definen virtualmente las posiciones del tablero
235     String row1 = "ABCDEFGHJKLMN";
236     String row2 = "NOPQRSTUVWXYZ";
237     String row3 = "0123456789+-";
238
239     // Si no se provee una letra, muestra la última posición (
240     if (letter == "") {
241         turnOn(led0_3);
242         turnOn(led13);
243     } else {
244
```

Serial Monitor X Output

Not connected. Select a board and a port to connect automatically.

Palabra similar: ALFA

Letras correctas: 2

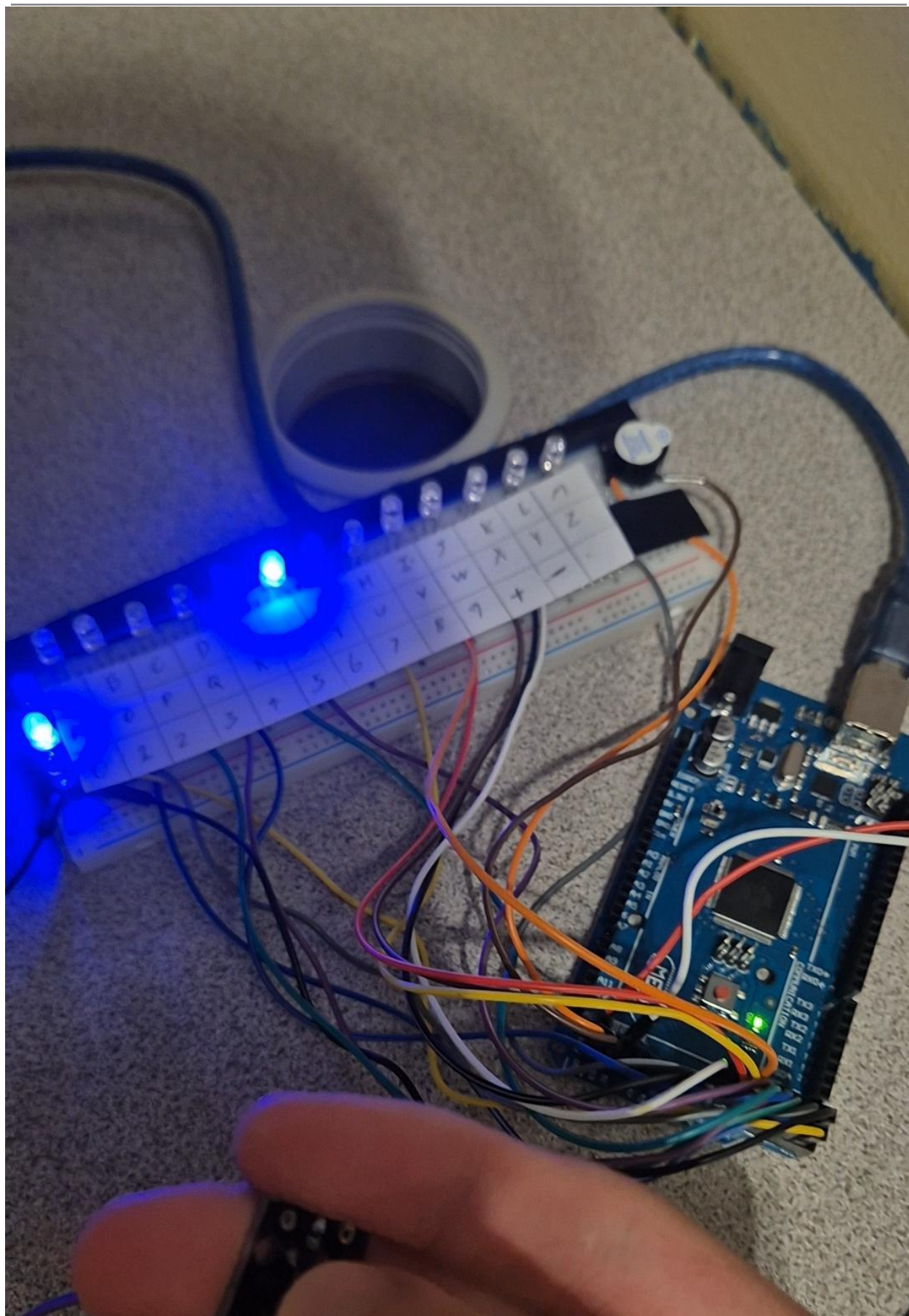
Estuviste cerca. Sigue intentándolo.

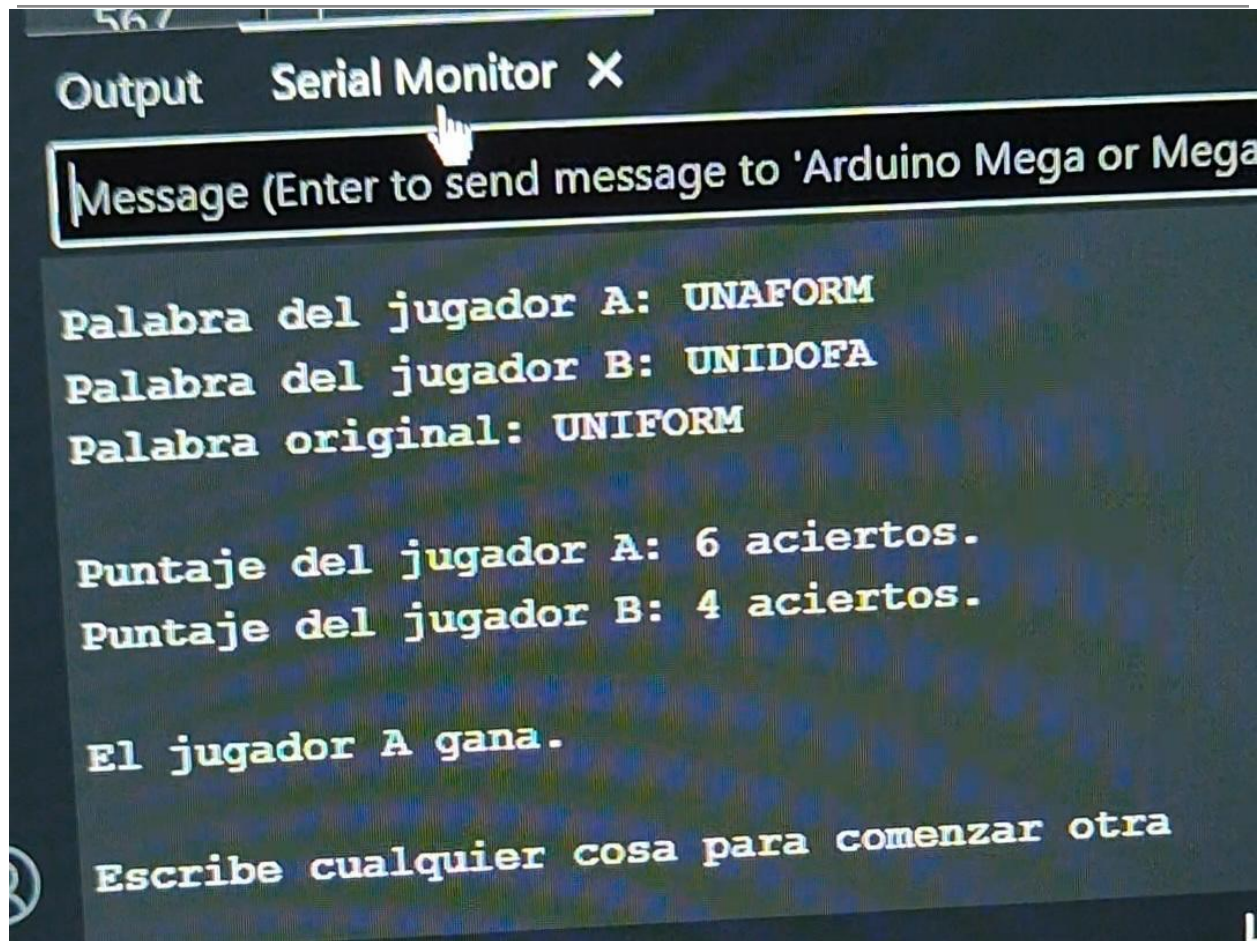
¿Qué deseas hacer ahora?.

1. Jugar otra partida.

2. Volver al menú principal.

>





Problemas encontrados:

- Problemas Solucionados:

- En CP-002, durante la ejecución de la prueba hubieron algunos LEDs que no encendieron mientras se ejecutaba la revisión inicial. La causa del problema estaba relacionada con cables sueltos y LEDs que no estaban correctamente fijados o que estaban colocadas incorrectamente. El problema fue corregido implementando cinta aislante y una banda de papel que mantiene los LEDs fijos en un sitio.
- Para CP-004, en un inicio la combinación de LEDs establecidas en el código no mostraba ningún resultado en el módulo físico. Esto se debió a un fallo con el shift register (74LS164) al implementar el breadboard adicional. Para solucionarlo, se revisó el cableado general del arduino. A pesar de presentar un reto debido al cambio de la estructura del prototipo, se consiguió solucionar el problema.
- En CP-006 algunos componentes del módulo eran inestables y por lo tanto frágiles al movimiento, principalmente los LEDs y resistencias, se

utilizó cinta aislante y papel para evitar que los componentes se muevan de su sitio. Esto último soluciona el problema.

- Problemas no Solucionados:
 - Durante la prueba CP-003, se determinó que el pulsador se encuentra funcional, pero en mal estado. Este mismo no detecta como es debido las pulsaciones, por lo que puede pasar que las pulsaciones no correspondan con los caracteres deseados. Dado que el pulsador sigue siendo funcional un gran porcentaje de las veces y que no se cuenta con un repuesto para el mismo, no se considera un problema crítico, y se deja pasar.
 - Existe un error identificado que ocurre esporádicamente en el cual es posible que en el monitor serial de Arduino IDE se impriman signos de pregunta (?), es posible incluso que dicho carácter se presente de forma repetida en textos de salida que sí están programados. El código no cuenta con ninguna salida pensada para que genere dicho resultado. Se tiene la teoría de que esto pueda deberse a desfases y fallos de comunicación entre el arduino y la computadora, provocando interrupciones que generan dicho símbolo repetidamente.

6. Declaración de uso de Inteligencia Artificial y herramientas

6.1. Herramientas de IA utilizadas

Herramienta de IA	Tipo	Período de Uso	Propósito
Gemini	Modelo de lenguaje (LLM)	04/2026	Sugerencia y explicación de problemas en el código
Microsoft Copilot	Modelo de lenguaje (LLM)	04/2026	Investigación y búsquedas referentes a temas muy específicos

6.2. Descripción del Uso de Inteligencia Artificial

- Gemini: Utilizada únicamente para mejorar y optimizar ciertas funcionalidades que no cumplían con un resultado esperado. Cabe aclarar que no se utilizó código generado por esta inteligencia artificial, su uso fue únicamente para idear y sugerir cambios en dichas funcionalidades previamente programadas. Su uso y aporte en el proyecto fue validado por ambas partes del equipo, y sus sugerencias fueron aplicadas según el contexto del código.
- Microsoft Copilot: Se utilizó principalmente como herramienta investigativa respecto a temas específicos como funcionamiento del arduino y relativo al desarrollo del presente documento. No se implementaron respuestas específicas en el documento sino su interpretación por parte del equipo por lo que fue validado y aprobado por ambas partes.

6.3. Declaración de Responsabilidad y Autoría

Los suscritos, integrantes del equipo de trabajo del proyecto “StrangerTEC”, declaramos que:

- El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de trabajo intelectual propio y esfuerzo en equipo.
- Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
- Todo el contenido generado por Inteligencia Artificial (IA) fue revisado, comprendido, validado y, cuando fue necesario, corregido por el equipo, para cumplir con la calidad de la asignatura.
- Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
- El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

Firman:



Ignacio José Apuy Anchía
Integrante del proyecto.



Jordanny Steven Hernández Mora
Integrante del proyecto.

7. Conclusión

A través de este proyecto se ha conseguido recrear un minijuego que simula el funcionamiento del muro de luces presentado en la serie Stranger Things. Se utilizaron en conjunto ambos elementos de hardware y software para cumplir con el objetivo. Además de esto se han implementado conceptos y estructuras del código Morse como idea central del minijuego para enfrentar a dos jugadores en este.

Los objetivos planteados para el proyecto se han superado. Se tiene un prototipo funcional que permite a los jugadores interactuar con este mismo para poner a prueba conocimientos en código Morse y sus habilidades observativas.

Como posibles mejoras futuras al prototipo actual se tiene pensado optimizar el espacio que ocupa este mismo, además de modos de juego con conectividad inalámbrica hacia otros prototipos similares. Diseñar un mejor contenedor que luzca llamativo y sea resistente y mejorar el repertorio de palabras y caracteres utilizados.

Mediante este proyecto se consiguieron nuevos conocimientos respecto al uso de arduino y el funcionamiento de la electrónica, con un entorno controlado que permite experimentar al usuario con esta misma. Además de ello se han adquirido nuevas técnicas para programación enfocada en C++ y en uso de componentes de hardware. Se han obtenido conocimientos introductorios respecto al funcionamiento del código Morse y cómo implementarlos en un programa.

Finalmente, la participación complementaria del equipo permitió diseñar un prototipo de tamaño ideal para el diseño externo de la maqueta. Además de las sugerencias que permitieron complementar el trabajo asignado a ambas partes, y colaboración en todo momento respecto a problemas con su parte.

8. Bibliografía

- [1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, IEEE Computer Society, New York, NY, USA, 1998. doi: 10.1109/IEEESTD.1998.88286
- [2] ISO/IEC, *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011, International Organization for Standardization, Geneva, Switzerland, 2011.
- [3] IEEE, *IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*, IEEE Std 1016-2009, IEEE Computer Society, New York, NY, USA, 2009. doi: 10.1109/IEEESTD.2009.5167255
- [4] Google LLC, *Google Cloud Architecture Framework*, Google Cloud Documentation, 2024. [En línea]. Disponible en: <https://cloud.google.com/architecture/framework>
- [5] Microsoft Corporation, *Azure Well-Architected Framework*, Microsoft Learn Documentation, 2024. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/azure/well-architected/>
- [6] Microsoft Corporation, *Microsoft Writing Style Guide*, Microsoft Docs, 2024. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/style-guide/welcome/>
- [7] Google LLC, *Google Developer Documentation Style Guide*, Google Developers, 2024. [En línea]. Disponible en: <https://developers.google.com/style>
- [8] ISO/IEC/IEEE, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, International Organization for Standardization, Geneva, Switzerland, 2018. doi: 10.1109/IEEESTD.2018.8559686
- [9] M. Nygard, *Architecture Decision Records (ADRs)*, ThoughtWorks Technology Radar, 2017. [En línea]. Disponible en: <https://adr.github.io>
- [10] Google LLC, *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, Sebastopol, CA, USA, 2016. [En línea]. Disponible en: <https://sre.google/sre-book/table-of-contents/>
- [11] Autodesk Instructables, *The 74HC164 Shift Register and Your Arduino*, osgeld. [En Línea]. Disponible en: <https://www.instructables.com/The-74HC164-Shift-Register-and-your-Arduino/>
- [12] Autodesk Instructables, *The Arduino Hang Guardian - Arduino Watchdog Timer Tutoria*, taste_the_code. [En línea].

Disponible en: <https://www.instructables.com/The-Arduino-Hang-Guardian-Arduino-Watchdog-Timer-T/>

[13] Arduino Docs, *randomSeed()*, Arduino. [En línea].

Disponible en: <https://docs.arduino.cc/language-reference/en/functions/random-numbers/randomSeed/>

[14] Circuits, *DIY, 74LS164 8-Bit Shift Register IC With Parallel Outputs – Datasheet*, Syed Saan Hasan, 2020. [En Línea].

Disponible en: <https://www.circuits-diy.com/74ls164-8-bit-shift-register-ic-with-parallel-outputs-datasheet/>

[15] Arduino Docs, *Arduino IDE*, Arduino. [En línea]

Disponible en: <https://docs.arduino.cc/software/ide/>

[16] Arduino Docs, *Mega 2560 Rev3*, Arduino. [En línea].

Disponible en: <https://docs.arduino.cc/hardware/mega-2560/>

[17] MorseCode.World, *Morse Code Timing*, C, Phillips. [En Línea]

Disponible en: <https://morsecode.world/international/timing/>